

AGENTIC WORKFLOW PARADIGM

IDEATION-AUGMENTED SPEC-DRIVEN DEVELOPMENT (IA-SDD)

Bidirectional Socratic Ideation and Empirical Verification in Agentic Workflows

THE OPERATIONAL DIVIDE: VIBE CODING VS. SDD

Source of Truth

VIBE CODING

Ephemeral context window, uncommitted chat

SPEC-DRIVEN DEV

Version-controlled Git artifacts (.openspec/, .specify/)

Interface Invariants

VIBE CODING

Malleable, emergent, untyped

SPEC-DRIVEN DEV

RFC 2119 imperatives, EARS requirements notation, other formal schemas

Verification Gate

VIBE CODING

Reactive debugging against runtime failures

SPEC-DRIVEN DEV

Deterministic CI linting, unit tests, and drift tests

Key Takeaway

Vibe coding builds against its own last prompt output; SDD builds against an immutable repository contract.

WHY VIBE CODING FAILS IN PRODUCTION SYSTEMS

01 / ATTENTION LOSS

Silent Context Drift

Context window expansion causes early constraints (thread safety, bounds, timeouts) to drop from attention weights.

02 / PROCESS BYPASS

Bypassed SDLC Phases

Eliminates formal architecture, interface design, and defensive error handling in favor of prompt-patch cycles.

03 / MISALIGNED OUTPUT

Unbounded Agent Drift

The agent confidently generates 500 lines of plausible code that solves the wrong problem.

04 / REPO INSTABILITY

Architectural Thrashing

Bug-fix prompts rewrite working patterns, swap dependencies, and break uncommitted contracts.

SPECIFICATION DRIVEN DEVELOPMENT

Core Definition & Purpose

SDD is an engineering process that defines specifications in a structured, human-readable format as a single source of truth **before any coding or AI code generation**.

01 / METHODOLOGY SHIFT

Eliminating Ad-Hoc "Vibe Coding"

Replaces unconstrained prompting and unstructured coding with explicit boundaries and deterministic rules for developers and AI agents.

02 / GOVERNANCE ARTIFACTS

Structured Control Stack

Controls software construction using standardized version controlled files:

- Markdown Specification Files
- Architecture & Execution Plans
- Granular Task Breakdowns

SPECIFICATION DRIVEN DEVELOPMENT: THE BLANK-SPEC PROBLEM

THE LITERATURE ASSUMPTION

Frameworks (OpenSpec, Spec Kit, Kiro) assume a complete, hardened specification already exists in Git.

The Real-World Bottleneck: Engineers suffer from the Blank-Spec Problem

01 / NEGATIVE CONSTRAINTS

Omission of Constraints

Forgetting what the system **MUST NOT** do during execution.

02 / EDGE CONDITIONS

Untyped Edge Cases

Relying on vague directives like 'handle retries gracefully'.

03 / ARCHITECTURE

Unvetted Guesses

Building on untested concurrency or caching models.

CORE THESIS

An under-specified contract forces the coding agent to hallucinate architectural invariants.

IDEATION-AUGMENTED SPEC-DRIVEN DEVELOPMENT (IA-SDD)

Three Distinct Execution Phases

01 / PHASE 1

Bidirectional Socratic Ideation

Socratic dialectic between architect and LLM exploring boundaries and failure domains.

02 / PHASE 2

Synthesis & Vetting Funnel (Human-Gated)

Compilation into RFC 2119 + Markdown + other formal structures.

Mandatory human dependency audit and isolated TDD spike.

Note: A spike is a quick, time-boxed, research experiment used to explore design ideas

03 / PHASE 3

Downstream Deterministic Execution

Agent executes strictly against frozen Git specification.

CI enforces continuous drift checks via git diff.

PHASE 1: BIDIRECTIONAL SOCRATIC IDEATION

Socratic Dialectic Between Architect and LLM Exploring Boundaries & Failure Domains

01 / LLM AS SOCRATIC INTERROGATOR

Probes Operational Boundaries

- Downstream backpressure and timeout tolerances
- At-least-once vs. exactly-once message semantics
- Idempotency key expiration and collision behavior
- Memory ceilings per tenant

02 / ENGINEER AS SOCRATIC QUESTIONER

Interrogates Model Abstractions

- Interrogates model's proposed abstractions and trade-offs
- Compares protocols (gRPC vs. WebSocket vs. REST)
- Looks for non-functional failure modes before selecting dependencies

PHASE 2: THE SYNTHESIS AND VETTING FUNNEL

Formalizing Machine-Readable Contracts & Enforcing Anti-Hallucination Gates

01 / CONTRACT SYNTHESIS

Spec & Schema Compilation

- **RFC 2119 Imperatives:** Strict normative language (MUST, MUST NOT, SHALL)
- **Concrete Data Contracts:** Strongly typed Pydantic v2 / Protobuf schemas
- **Behavioral Verification:** Explicit Gherkin Syntax or Code-First approaches

02 / MANDATORY HUMAN VETTING

Anti-Hallucination

- **Dependency Audit:** Verify every library, SDK method, and CLI flag against official documentation.
- **Empirical TDD Spiking:** Extract high-risk mechanics into prototypes and test fixtures; verify execution before committing the spec.

ARTIFACT STRUCTURE: RFC 2119 & TYPED SCHEMAS

Machine-Readable Normative Rules & Strict Data Type Definitions

01 / NORMATIVE CONSTRAINTS

.openspec/webhook_engine.md (example)

- **1.1 Delivery:** SHALL guarantee at-least-once delivery.
- **1.2 Timeouts:** MUST enforce `httpx.Timeout(connect=2.0, read=5.0, pool=10.0)`.
- **1.3 Security:** MUST include HMAC-SHA256 signature header.
- **1.4 Concurrency:** MUST NOT exceed 20 concurrent open sockets per destination host.
- **1.5 Retry Backoff:** Failed dispatches SHALL compute retry backoff via `min(3600, 5 * (2 ** attempts))` and record to Redis ZSET.
- **1.6 In-Memory State:** MUST NOT maintain retry sleep state in process memory.

02 / TYPED SCHEMA

Data Contracts

- **Pydantic v2 Contract:** Schemas with strict validation bounds.
- **Type Enforcement:** Machine-verifiable runtime constraints preventing payload corruption.
- **Zero Ambiguity:** Precise property types and mandatory field declarations.

PHASE 3: DOWNSTREAM EXECUTION AND DRIFT CONTROL

Automated Code Generation, Continuous Spec Compliance & Controlled Iteration

01 / EXECUTION

Deterministic Agent Execution

- **Spec-Bound Agents:** Coding agents (Cursor, OpenCode CLI, Claude Code) read only the committed spec.
- **Test Suites:** Generates unit test suites directly preferred method of testing.

02 / VERIFICATION

Continuous Drift Verification

- **Version Control:** Specifications are committed and tracked directly in version control.
- **Automated CI filters:** CI fails via automated linting if code changes decouple from spec invariants.

03 / ITERATION

Delta Specifications

- **Modular Evolution:** Subsequent features are drafted as delta specs following the same Socratic funnel.
- **Zero Regression:** Prevents feature drift by isolating incremental modifications.

EVALUATION MATRIX: THREE ENGINEERING MODES

Comparative Analysis Across Key Software Engineering Dimensions

01 / VIBE MODE

Vibe Coding

- **Requirements:**
Reactive bugs
- **Contract Rigor:**
None (chat)
- **Hallucination Defense:**
Ad-hoc debugging
- **Drift Resistance:**
Near-zero
- **Production Suitability:**
Toy scripts

02 / RAW SDD

Raw SDD

- **Requirements:**
Assumes completeness
- **Contract Rigor:**
Markdown / EARS
- **Hallucination Defense:**
Minimal
- **Drift Resistance:**
High (Git)
- **Production Suitability:**
Standard tasks

03 / IA-SDD (OPTIMAL)

IA-SDD Framework

- **Requirements:**
Socratic dialogue
- **Contract Rigor:**
RFC 2119 + Schemas +
Markdown
- **Hallucination Defense:**
Human SDK audit + TDD spike
- **Drift Resistance:**
Absolute (CI drift checks)
- **Production Suitability:**
Mission-critical systems

ENGINEERING PROTOCOLS: FIVE ACTIONABLE RULES

Mandatory Guardrails for Spec-Driven Agentic Software Engineering

01 Partition Ideation from Mutation: Brainstorm in chat; coding agents touch repositories only via committed specs.

02 Never Commit an Unvetted Dependency: Audit every external library and API against official documentation.

03 Spike High-Risk Mechanics with TDD: Write a prototype and tests to prove runtime behavior prior to finalizing the spec.

04 Enforce Negative Constraints & Schemas: Mandate RFC 2119 clauses ('MUST NOT') and strong types definitions.

05 Freeze the Spec Before Code Generation: Code is merely a downstream compilation target of the specification.

DISCUSSION & REFERENCES

Academic Papers, Industry & Technical Literature

01 IBM Technology: 'Spec-Driven Development: AI Assisted Coding Explained', 2026.

02 Use AI with Tech Dad: 'OpenSpec vs Spec Kit vs Kiro: Which Spec-Driven Workflow Should You Actually Use?', 2026.

03 D. B. Piskala: 'Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants', arXiv:2602.00180, 2026.

04 J. Diaz et al.: 'Spec-Driven Development for Agentic Software Engineering: Harnessing Human-Agent Teamwork', arXiv:2609.00252, 2026.